LOCAL SEARCH ALGORITHMS

From path-finding to state-improving

- Uninformed and informed search (UCS, BFS, DFS, Greedy, A*, etc.) explore a space of paths - the goal is to find a sequence of actions from start to goal.
- Local search takes a different approach: it works with a single current state and tries to improve it, without keeping track of paths or how it got there.

Key idea:
Start with a state (a possibly poor solution), and move to better neighbouring states.

Why use local search?

- Many problems only care about the final configuration, not the path (for example, job scheduling, robot path planning, hyperparameter tuning, circuit design, Travelling Salesman Problem, games/puzzles such as 8-queens, etc.).
- Uses very little memory - just the current state (and maybe a few others).
- Can find reasonable solutions in huge or even infinite state spaces, where systematic search is impractical.

Trade-off:

Usually does not explore the state space completely and may not return an optimal solution; but is often fast and practical.

Visualizing Local Search: The State Space Landscape

Think of every possible state as a point on a **landscape**:

Location = the state

Elevation = value of the objective/heuristic function at that state

Two ways to define "elevation":

Objective function - to be *maximized* → higher is better → think of **peaks**

Heuristic/cost function - to be *minimized* → lower is better → think of **valleys**

Goal of local search:

Find the **highest peak** (or **lowest valley**) - the *global maximum* (or *minimum*) - by moving to better neighbouring states.

Hill Climbing Search Algorithm (Greedy Local Search)

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

    current ← Initial state

    while true **do**:

        neighbour ← a highest-valued successor state of *current*

        if VALUE(*neighbour*) ≤ VALUE(*current*) **then return** *current*

        current ← *neighbour*

Illustration of hill climbing for the 8-Queens problem

- Let the state space be the set of all states having 8 queens on the board, one per column. Total number of states in the state space is $8^8 = 16{,}777{,}216$.

- For a given state n, let the successors be defined as all the states that can be obtained from n by moving a single queen to another square in the same column. Therefore, every state has $7 + 7 + 7 + 7 + 7 + 7 + 7 + 7 = 56$ successor states.

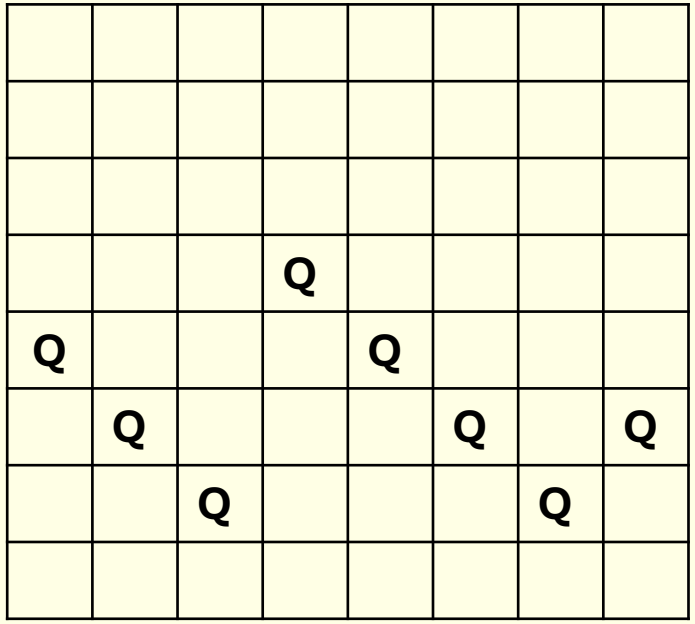
- Let the objective function be defined as the **negative** of the following heuristic function:
  $h(n) =$ Number of attacking pairs of queens in state n.

- Hence, maximizing the objective function is equivalent to minimizing $h(n)$.
  (Note that any solution to 8-queens problem has $h(n) = 0$.)

A sample iteration

We can represent any state as an 8-element array, where the ith element indicates the row number in which the queen in the ith column is ($1 \leq i \leq 8$).

Suppose the initial state is the following:



Array representation: [5,6,7,4,5,6,7,6]
$h(n) = 17$

The following are the 56 successors of the given initial state with respective values of $h(n)$:

[1,6,7,4,5,6,7,6] $h(n) = 18$	[5,1,7,4,5,6,7,6] $h(n) = 12$	[5,6,1,4,5,6,7,6] $h(n) = 14$	[5,6,7,1,5,6,7,6] $h(n) = 13$	[5,6,7,4,1,6,7,6] $h(n) = 13$	[5,6,7,4,5,1,7,6] $h(n) = 12$	[5,6,7,4,5,6,1,6] $h(n) = 14$	[5,6,7,4,5,6,7,1] $h(n) = 14$
[2,6,7,4,5,6,7,6] $h(n) = 14$	[5,2,7,4,5,6,7,6] $h(n) = 16$	[5,6,2,4,5,6,7,6] $h(n) = 13$	[5,6,7,2,5,6,7,6] $h(n) = 15$	[5,6,7,4,2,6,7,6] $h(n) = 12$	[5,6,7,4,5,2,7,6] $h(n) = 14$	[5,6,7,4,5,6,2,6] $h(n) = 12$	[5,6,7,4,5,6,7,2] $h(n) = 16$
[3,6,7,4,5,6,7,6] $h(n) = 14$	[5,3,7,4,5,6,7,6] $h(n) = 12$	[5,6,3,4,5,6,7,6] $h(n) = 18$	[5,6,7,3,5,6,7,6] $h(n) = 13$	[5,6,7,4,3,6,7,6] $h(n) = 15$	[5,6,7,4,5,3,7,6] $h(n) = 12$	[5,6,7,4,5,6,3,6] $h(n) = 14$	[5,6,7,4,5,6,7,3] $h(n) = 14$
[4,6,7,4,5,6,7,6] $h(n) = 15$	[5,4,7,4,5,6,7,6] $h(n) = 14$	[5,6,4,4,5,6,7,6] $h(n) = 14$	[5,6,7,5,5,6,7,6] $h(n) = 15$	[5,6,7,4,4,6,7,6] $h(n) = 13$	[5,6,7,4,5,4,7,6] $h(n) = 16$	[5,6,7,4,5,6,4,6] $h(n) = 13$	[5,6,7,4,5,6,7,4] $h(n) = 16$
[6,6,7,4,5,6,7,6] $h(n) = 17$	[5,5,7,4,5,6,7,6] $h(n) = 14$	[5,6,5,4,5,6,7,6] $h(n) = 17$	[5,6,7,6,5,6,7,6] $h(n) = 18$	[5,6,7,4,6,6,7,6] $h(n) = 15$	[5,6,7,4,5,5,7,6] $h(n) = 14$	[5,6,7,4,5,6,5,6] $h(n) = 16$	[5,6,7,4,5,6,7,5] $h(n) = 16$
[7,6,7,4,5,6,7,6] $h(n) = 18$	[5,7,7,4,5,6,7,6] $h(n) = 14$	[5,6,6,4,5,6,7,6] $h(n) = 16$	[5,6,7,7,5,6,7,6] $h(n) = 15$	[5,6,7,4,7,6,7,6] $h(n) = 15$	[5,6,7,4,5,7,7,6] $h(n) = 14$	[5,6,7,4,5,6,6,6] $h(n) = 15$	[5,6,7,4,5,6,7,7] $h(n) = 16$
[8,6,7,4,5,6,7,6] $h(n) = 14$	[5,8,7,4,5,6,7,6] $h(n) = 14$	[5,6,8,4,5,6,7,6] $h(n) = 13$	[5,6,7,8,5,6,7,6] $h(n) = 17$	[5,6,7,4,8,6,7,6] $h(n) = 12$	[5,6,7,4,5,8,7,6] $h(n) = 14$	[5,6,7,4,5,6,8,6] $h(n) = 12$	[5,6,7,4,5,6,7,8] $h(n) = 18$

The least value of $h(n)$ (that is, the highest value of the objective function) among the successor states is 12. The algorithm chooses one of the successor states with this value of $h(n)$ as *neighbour*, say, the state [5,3,7,4,5,6,7,6].

The algorithm continues in this way until it finds a state such that none of its successors have a better value than itself, and then returns that state.

The following is a possible sequence of states chosen in five successive iterations in this example:

$[5,3,7,4,5,6,7,6]$ with $h(n) = 12$

$[5,3,7,4,2,6,7,6]$ with $h(n) = 7$

$[5,3,7,4,2,6,1,6]$ with $h(n) = 4$

$[8,3,7,4,2,6,1,6]$ with $h(n) = 3$

$[8,3,7,4,2,5,1,6]$ with $h(n) = 1$

The successors of the state [8,3,7,4,2,5,1,6] have heuristics varying between 2 and 5.

The algorithm chooses one of the successor states with $h(n) = 2$ as *neighbour*, say, the state [8,3,7,4,2,5,1,4].

As $h([8,3,7,4,2,5,1,4]) > h([8,3,7,4,2,5,1,6])$, the algorithm returns [8,3,7,4,2,5,1,6] and terminates. (There is no better-valued successor.)

However, [8,3,7,4,2,5,1,6] is a local minimum (local maximum in terms of the objective function); because a global minimum (a solution to the 8-queens problem) has $h(n) = 0$.

Therefore, hill climbing can get stuck in a local maximum.

The problem with hill-climbing

- Hill-climbing only ever moves to a **better** neighbouring state - it never accepts a move that makes things worse.

- Because of its "greedy" behaviour, it can easily get stuck in local optima or plateaus (flat regions with no strictly better neighbour).

Once stuck, hill-climbing has no way to escape - it simply stops.

The key idea behind simulated annealing

What if, instead of *always* rejecting worse moves, we *sometimes* allowed them? Occasionally accepting a downhill move gives the search a chance to move through local optima or plateaus, instead of stopping at them.

How often should we accept a worse move?
Too often: The search behaves like a random walk, never settling down.
Too rarely: It behaves just like hill-climbing, can get stuck as before.

The balance simulated annealing strikes:
Accept worse moves **more freely early on**, and **less freely as time goes on –** This is controlled by a gradually decreasing **temperature** parameter.
Inspired by the physical process of *annealing* in metallurgy - slowly cooling a material to let it settle into a low-energy, stable structure.

Simulated Annealing Algorithm

function SIMULATED-ANNEALING(*problem*) **returns** a solution state

current ← Initial state

for $t = 1$ to t_{max} **do**:

$T \leftarrow schedule(t)$

if $T = 0$ **then return** *current*

next ← a randomly chosen successor of *current*

if VALUE(*next*) > VALUE(*current*) **then**

current ← *next*

else

$\Delta E \leftarrow$ VALUE(*current*) − VALUE(*next*)

$prob \leftarrow e^{-\frac{\Delta E}{T}}$

if *random()* < *prob* **then** *current* ← *next*

Note:

1. *schedule* is a decreasing function of time t.
2. *random()* stands for a random real number in $[0,1)$. The last line of the pseudocode is to update *current* to *next* with probability *prob*.

- As $\Delta E \to 0$, $prob \to 1$.
  Small downhill moves ($\Delta E \to 0$) are **almost always accepted**, since they represent only a slightly worse state (worth risking to escape shallow local optima or plateaus).

- As $\Delta E \to \infty$, $prob \to 0$.
  Large downhill moves ($\Delta E \to \infty$) are **almost never accepted**, since jumping to a much worse state is rarely worth it.

- As $T \to 0$, $prob \to 0$.
  As the temperature cools ($T \to 0$), that is, at later iterations, the algorithm becomes **increasingly reluctant to accept any downhill move**, gradually behaving like plain hill-climbing.

We *explore* freely early on (when T is high), and settle down and *exploit* the existing information later on (when T gets lower).

(*Exploration–exploitation dilemma / Explore-exploit tradeoff)

Note: A possible modification to simulated annealing algorithm is to terminate as soon as a solution is found, without going through the remaining iterations.
(This can be done if the best value of the objective function is known beforehand.)

A Sample Run of Simulated Annealing

Initial state

[8, 3, 7, 4, 2, 5, 1, 6]

h = 1 (1 attacking pair of queens)

This is the same local minimum shown in the hill-climbing example. Every single-queen move from here is worse - hill climbing stops here.

What lets Simulated Annealing escape it:

Accepts a worse move with probability **exp(-ΔE / T)**.

T starts high (freely explores) and cools over time (settles down)

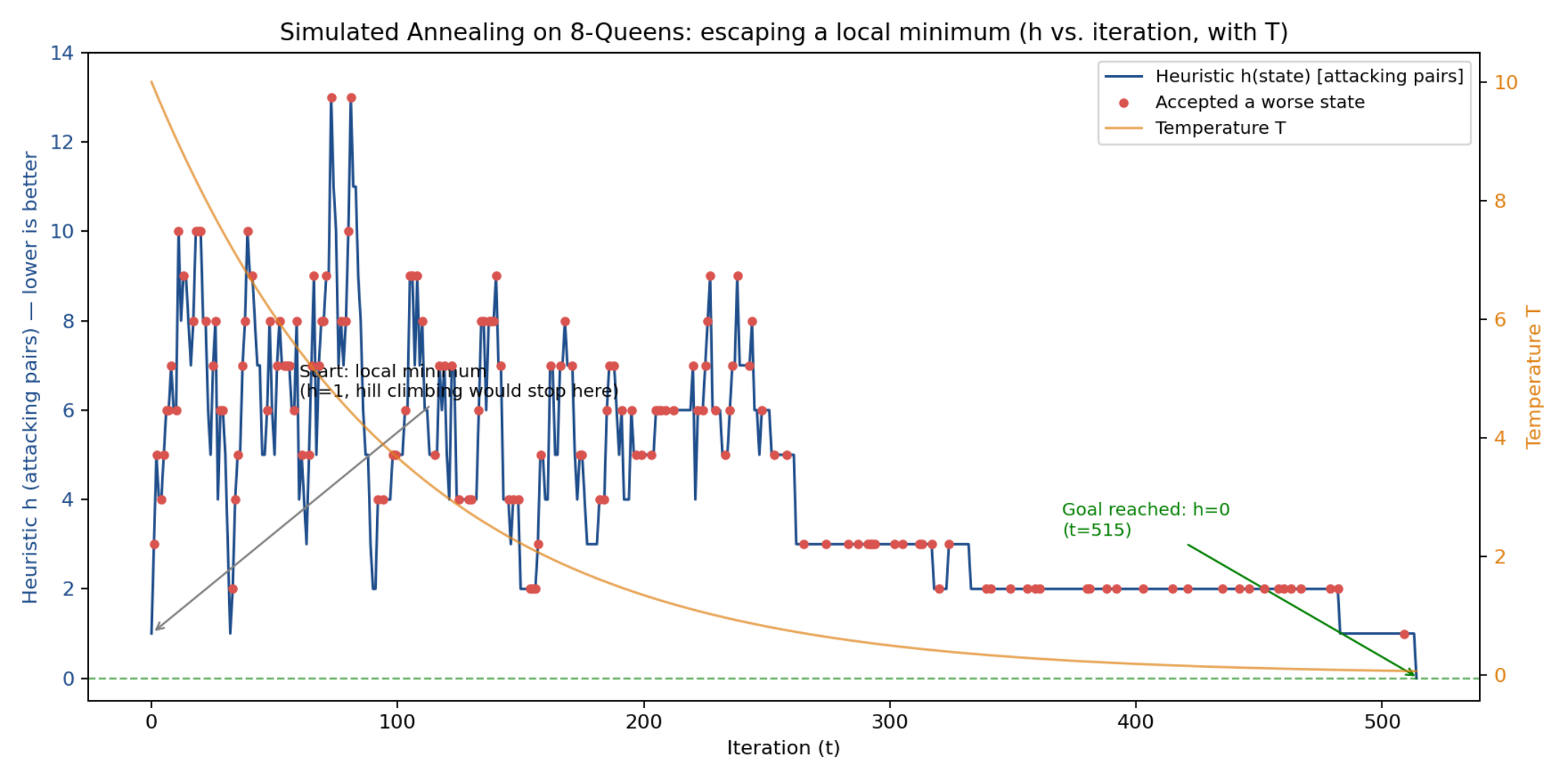
This run

Cooling schedule	**T = 10 × 0.99^t**
Max iterations	**5000**
Iterations actually used	**515**
Worse moves accepted	**156**
Improving moves made	**71**
Final heuristic	**h = 0 (solution found)**

Simulated Annealing is stochastic — this is one possible run. A different run may take a different path or take more/fewer iterations to converge.

The full trajectory

Heuristic h (attacking pairs) and temperature T over 515 iterations



From single-solution search to population-based search

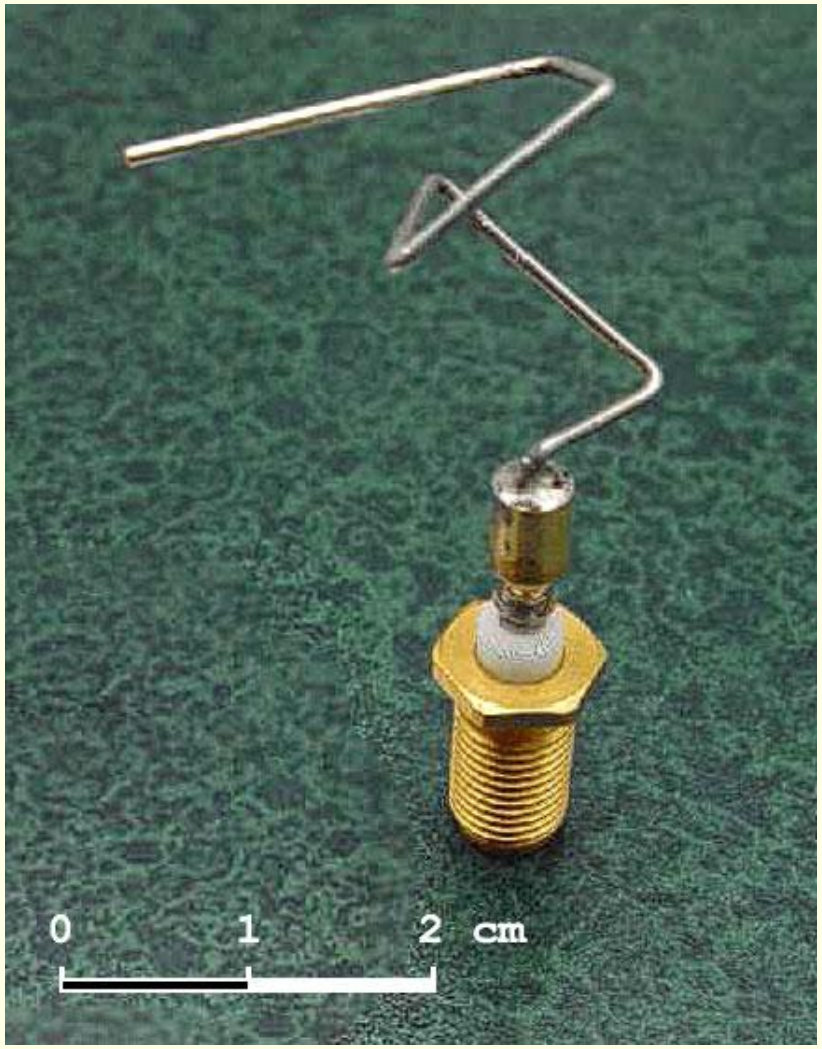
- Simulated annealing is a single-solution search; it maintains one candidate state at a time.
- This single-solution search can get stuck exploring one region, even with random restarts.

Idea: What if we searched with a whole *population* of states in parallel?

- Diversity across a population leads to a broader coverage of the search space.
- Good sub-solutions found in one state could be combined with good sub-solutions from another state.

This is the core idea behind **Evolutionary Algorithms**: Bring biological evolution (variation and selection) into optimization.

The 2006 NASA ST5 spacecraft antenna



This antenna shape was found using an evolutionary algorithm to create the best radiation pattern.

Image source: X-band Antenna, Space Technology 5 (ST-5), Wikimedia Commons (Public Domain, NASA). https://commons.wikimedia.org/wiki/File:St_5-xband-antenna.jpg

Reference:
Lohn, J.D., Hornby, G.S., Linden, D.S. (2005). An Evolved Antenna for Deployment on Nasa's Space Technology 5 Mission. In: O'Reilly, UM., Yu, T., Riolo, R., Worzel, B. (eds) Genetic Programming Theory and Practice II. Genetic Programming, vol 8. Springer, Boston, MA. https://doi.org/10.1007/0-387-23254-0_18

Evolutionary Algorithms

An evolutionary algorithm is a population-based, stochastic optimization/search method inspired by natural selection.

Core loop (per generation):

1. Initialize a population of candidate solutions (states).
2. Evaluate the fitness of each state in the population.
3. Select fitter candidates to reproduce.
4. Apply variation operators (recombination, mutation) to produce offspring.
5. Form the next generation.

This is repeated until a stopping criterion is met.

With a reasonably large initial population (at least 100 or 200 for medium-level problems), over a large number of generations (usually 500 – 1000 generations), the algorithm tries to ensure *"survival of the fittest"*, thereby leading to a better solution.

Key Components of an Evolutionary Algorithm

Representation: Encoding a candidate solution/state (string, real vector, tree, etc.).

Fitness function: A function that measures the quality of a solution/state.
(Example: Negative of a heuristic function)

Selection mechanism: To determine which individuals reproduce.
(Examples: Weighted-Random-Choices / Roulette wheel selection, Tournament selection)

Variation operators: Crossover/Recombination, Mutation.

One of the types of evolutionary algorithms is **Genetic Algorithms**.

In a genetic algorithm, each state is represented as a string over a finite alphabet.

Inspired by genetics: Genes, crossover, mutation.

Hyperparameters to be chosen before running the genetic algorithm

1. **Size of the population**
2. **Fitness function**
3. **Mixing number**: The number of parent states used to produce a successor state (usually chosen as 2).
4. **Selection mechanism**: Commonly used methods are Weighted-Random-Choices function (also called Roulette wheel selection) and Tournament selection.
5. **Crossover point**: A randomly chosen number between 1 and n, where n is the length of the string representation of states.
6. **Mutation rate**: A small probability (about 0.05 or 0.1) with which some characters in an offspring are changed. This is used to ensure diversity over a large number of generations.

A sample iteration for 8-queens problem

Hyperparameters

Population size: 4

Fitness function: Number of non-attacking pairs of queens

Mixing number: 2

Selection mechanism: Roulette wheel selection

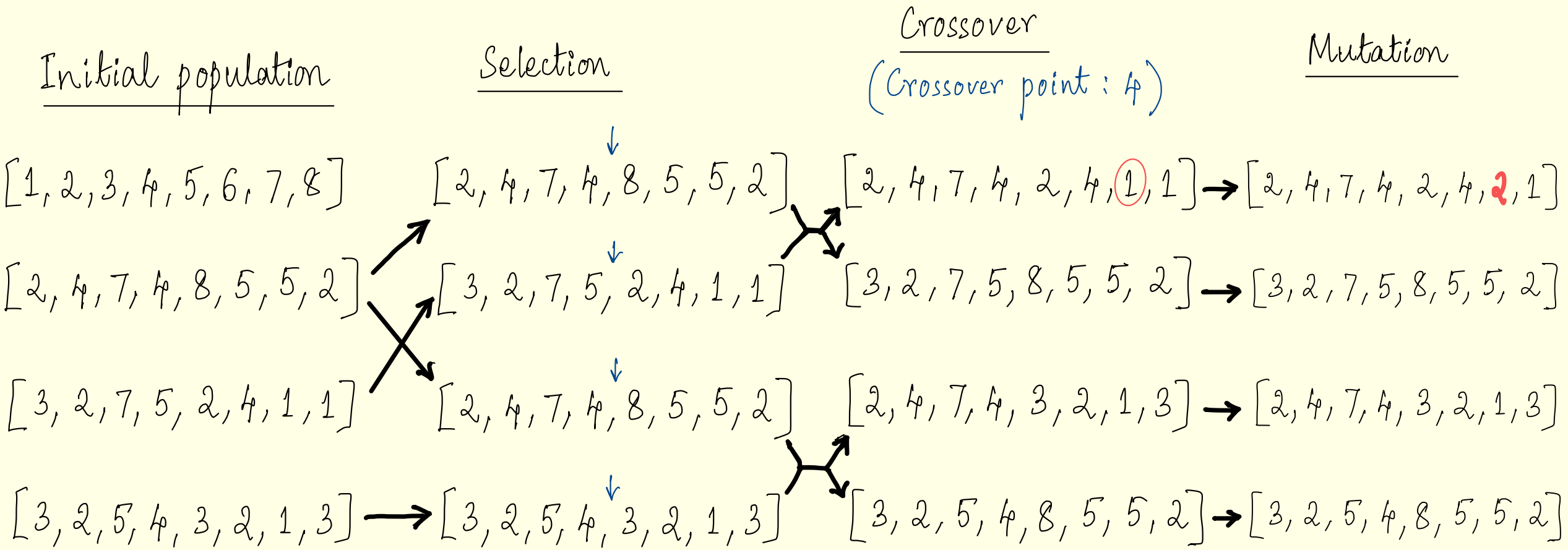
Crossover point: 4

Mutation rate: 0.1

Suppose the initial population is [1, 2, 3, 4, 5, 6, 7, 8], [2, 4, 7, 4, 8, 5, 5, 2], [3, 2, 7, 5, 2, 4, 1, 1] and [3, 2, 5, 4, 3, 2, 1, 3].

Initial population	Fitness scores	Probability of being chosen
[1, 2, 3, 4, 5, 6, 7, 8]	0	$\frac{0}{58} = 0$
[2, 4, 7, 4, 8, 5, 5, 2]	24	$\frac{24}{58} = 0.41$
[3, 2, 7, 5, 2, 4, 1, 1]	23	$\frac{23}{58} = 0.39$
[3, 2, 5, 4, 3, 2, 1, 3]	11	$\frac{11}{58} = 0.18$

Average fitness of the initial population $= 14.5$.


Initial population
Selection
Crossover
(Crossover point : 4)
Mutation
[1, 2, 3, 4, 5, 6, 7, 8]
[2, 4, 7, 4, 8, 5, 5, 2]
[2, 4, 7, 4, 2, 4, 1, 1] → [2, 4, 7, 4, 2, 4, 2, 1]
[2, 4, 7, 4, 8, 5, 5, 2]
[3, 2, 7, 5, 2, 4, 1, 1]
[3, 2, 7, 5, 8, 5, 5, 2] → [3, 2, 7, 5, 8, 5, 5, 2]
[3, 2, 7, 5, 2, 4, 1, 1]
[2, 4, 7, 4, 8, 5, 5, 2]
[2, 4, 7, 4, 3, 2, 1, 3] → [2, 4, 7, 4, 3, 2, 1, 3]
[3, 2, 5, 4, 3, 2, 1, 3] → [3, 2, 5, 4, 3, 2, 1, 3]
[3, 2, 5, 4, 8, 5, 5, 2] → [3, 2, 5, 4, 8, 5, 5, 2]

New generation	Fitness scores
[2, 4, 7, 4, 2, 4, 2, 1]	20
[3, 2, 7, 5, 8, 5, 5, 2]	22
[2, 4, 7, 4, 3, 2, 1, 3]	19
[3, 2, 5, 4, 8, 5, 5, 2]	20

Average fitness of the population = 20.25 (> 14.5).